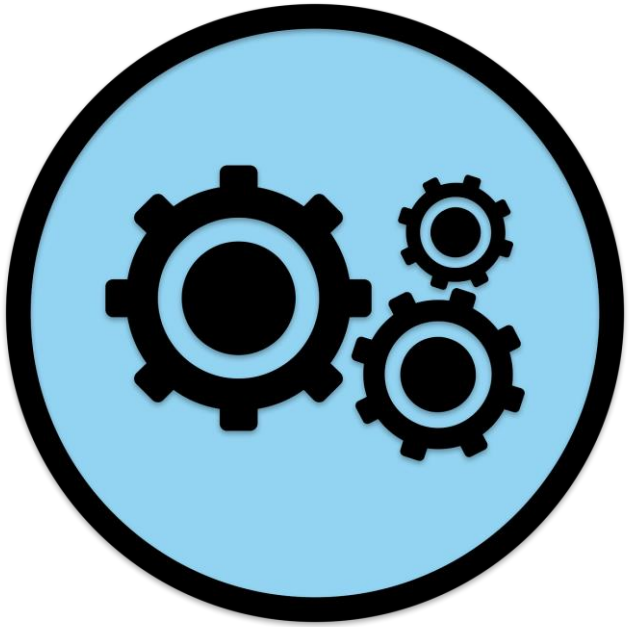
Technische Hochschule Ingolstadt
Fakultät Informatik

Software Engineering

Computer Science and Artificial Intelligence (B.Sc.)

Winter Term 2025/26

Chapter 2: Software-Quality



Prof. Dr.-Ing. Anatoli Djanatliev
Professorship for Software Development and Web Technologies

University of Applied Sciences Ingolstadt
Faculty of Computer Science

Learning objectives

- Being able to explain software quality in general
- Being able to describe quality characteristics according to ISO 25010
- Being able to name quality measures and indicators

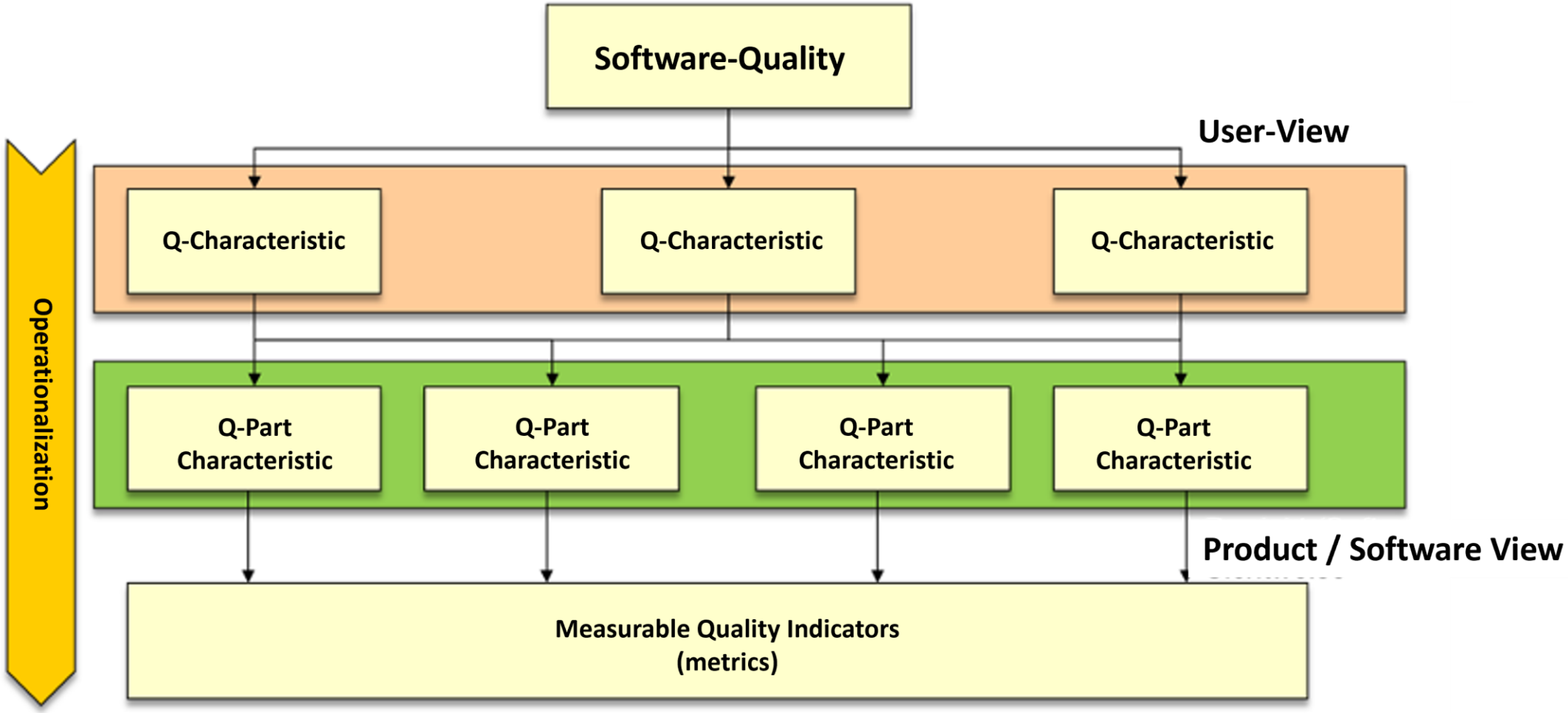
Content Table

2.1 Introduction

What is quality?

- **Definition of Quality:**
  - The totality of properties and characteristics of a unit (i.e. a product or an activity) with regard to its suitability to fulfill specified and assumed requirements.
- The term software quality based on the definition is usually not practicable in practice
  - … as measurable quality characteristics are required to assess the quality of software.
  - Models are needed that operationalize the concept of quality and make it assessable.
- The following approaches exist:
  - Predefined quality model, e.g. DIN ISO 25010 (formerly ISO/IEC 9126 or DIN 66272)
  - Development of individual quality models (e.g. GQM: goal - question - metric)
- The approaches work according to the principle of **making quality measurable**. This is the only way to achieve an objective assessment.

General structure of a quality model

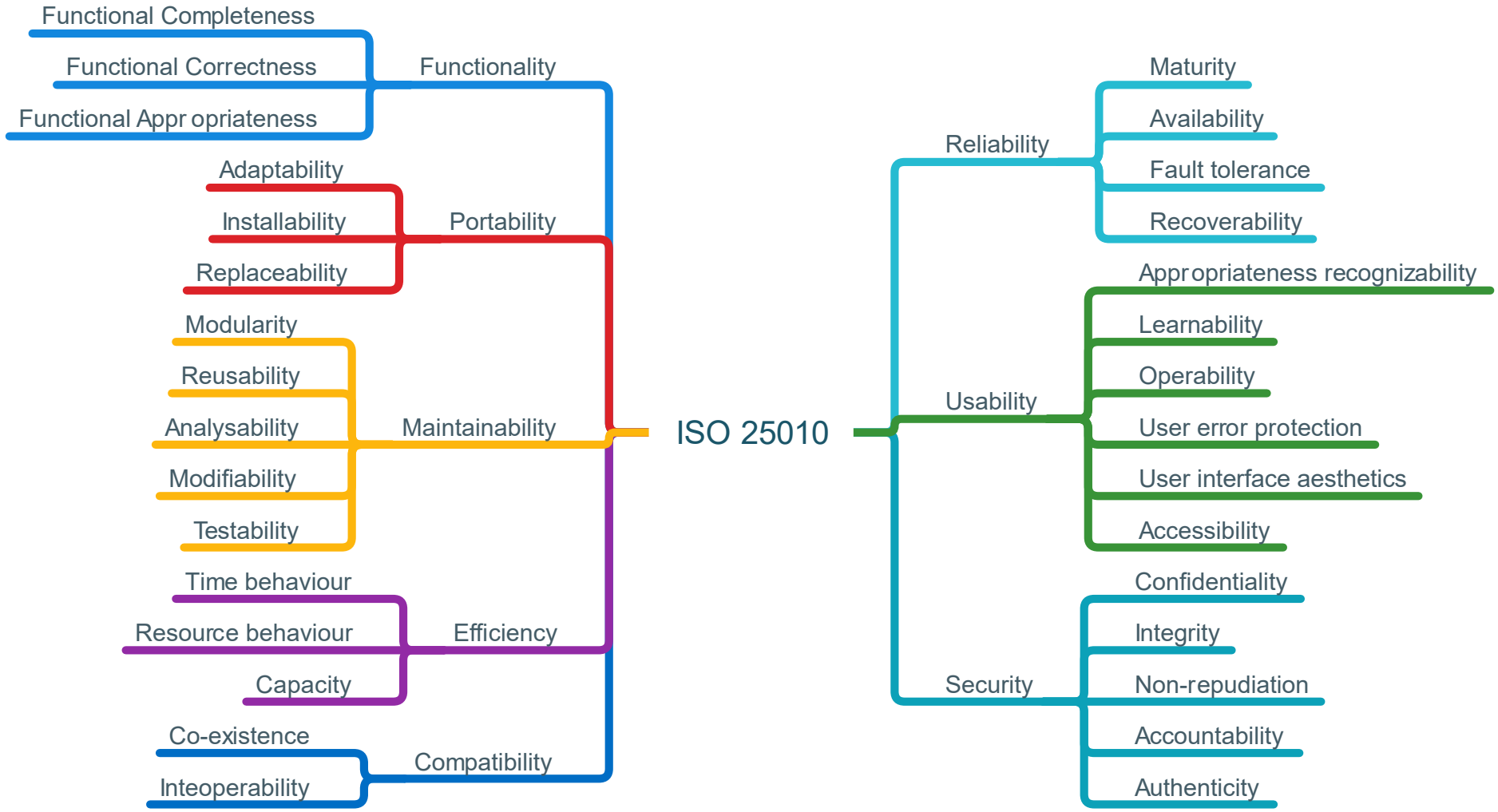


General structure of a quality model

- The graph shows the general structure of a quality model.
- Measurable quality indicators are defined for each quality sub-characteristic
- During the project phase, the software quality of the product can be continuously measured and improved through suitable measures.

2.2 Quality characteristics according to ISO 25010

ISO 25010 defines eight quality characteristics for SW products



Functionality

- Describes the extent to which a product or system provides functions that meet the stated and implied needs when used under certain conditions.
- The aspects are:
  - **Functional Completeness**:
    - Degree to which the set of functions covers all specified tasks and user goals.
  - **Functional Correctness**:
    - Degree to which a product or system delivers the correct results with the required degree of precision.
  - **Functional Appropriateness**:
    - Degree to which the functions facilitate the completion of the specified tasks and objectives.

Reliability

- This refers to the ability of the software to maintain its performance level under defined conditions over a defined period of time.
- The aspects are:
  - **Maturity**:
    - System, product or component meets the requirements for reliability in normal operation
  - **Availability**:
    - A system, product or component is operational and accessible when required for use.
  - **Fault Tolerance**:
    - A system, product or component can function as intended despite the presence of hardware or software faults.
  - **Recoverability**:
    - A product or system can recover the directly affected data and the desired state of the system in the event of an interruption or failure.

Usability

- Usability refers to the effort required for use and the individual assessment of use by a defined or assumed user group.
- The aspects are:
  - **Appropriateness Recognizability**:
    - Effort for the user to recognize whether a product or system is suitable for their own needs.
  - **Learnability**:
    - Effort for the user to learn the application
  - **Operability**:
    - Effort for the user to operate the application
  - **User error protection**:
    - The degree to which a system protects users from errors.

Usability

- **User interface aesthetics:**
  - The degree to which a user interface enables a pleasant and satisfying interaction for the user.
- **Accessibility**:
  - The degree to which a product or system can be used by people with a wide range of characteristics and abilities in order to achieve a specific goal in a specific context of use.

Efficiency

- Efficiency is defined as the relationship between the performance level of the software and the scope of the resources used under defined conditions.
- The aspects are:
  - **Time Behavior:**
    - Response and processing times as well as throughput during function execution.
  - **Resource Behavior:**
    - Number of resources required to fulfill the function and duration of resource use
  - **Capacity**:
    - Degree to which the maximum limits of a product or system parameter meet the requirements.

Maintainability

- This refers to the effort with which a product or system can be changed, improved or corrected, as well as changes in the environment or in the requirements can be implemented.
- The aspects are:
  - **Modularity:**
    - A system or computer program consists of individual components, so that a change to one component has only a minimal effect on other components.
  - **Reusability**:
    - The degree to which an asset can be used in more than one system or in the construction of other assets.

Maintainability

- **Analyzability**:
  - The degree of effectiveness and efficiency with which it is possible to evaluate the effects of an intended change to one or more parts of a product or system, to examine a product for defects or causes of defects, or to identify parts to be changed.
- **Modifiability:**
  - The degree to which a product or system can be effectively and efficiently modified without introducing defects or compromising existing product quality.
- **Testability:**
  - The degree of effectiveness and efficiency with which test criteria for a system, product or component can be defined and tests performed to determine whether these criteria have been met.

Portability

- Portability refers to the suitability of the software to be transferred from one environment to another.

The environment can include an organizational environment, hardware or software environment.

- The aspects are:
  - **Adaptability:**
    - The effectiveness and efficiency with which a product or system can be adapted to different or evolving hardware, software or other operating or usage environments.
  - **Installability:**
    - The effectiveness and efficiency with which a product or system can be successfully installed and/or uninstalled in a given environment.
  - **Replaceability:**
    - A product can be replaced by another specified software product for the same purpose in the same environment.

Compatibility

- Compatibility means the ability of the software to exchange information with other products, systems or components and/or to perform its required functions while using the same hardware or software environment.
- The aspects are:
  - **Co-existence:**
    - The degree to which a product can efficiently perform its required functions while sharing a common environment and resources with other products without adversely affecting another product.
  - **Inteoperability:**
    - The degree to which two or more systems, products or components can exchange information and utilize the information exchanged.

Security

- Describes that a product or system protects information and data so that people or other products or systems are given the level of data access appropriate to their nature and level of authorization.
- The aspects are:
  - **Confidentiality:**
    - A product or system ensures that data is only accessible to those who are authorized to access it.
  - **Integrity:**
    - A system, product or component prevents unauthorized access to or modification of applications or data.

Security

- **Non-repudiation:**
  - The degree to which actions or events can be proven to have taken place so that the events or actions cannot later be refuted.
- **Accountability:**
  - Degree to which the actions of an entity can be clearly traced back to that entity.
- **Authenticity:**
  - Degree to which it can be proven that the identity of a subject or a resource corresponds to the claimed identity.

Comment on the quality characteristics:

- Some of the quality characteristics are in conflict to each other
  - e.g. usability and efficiency or resource consumption and time efficiency
- The following quality characteristics are particularly relevant for end users:
  - reliability, user-friendliness, compatibility, efficiency
- The following quality characteristics are particularly relevant for software manufacturers:
  - reusability, portability
- The following quality characteristics are particularly relevant for manufacturers and buyers:
  - extensibility, customizability

2.3 Quality Measures

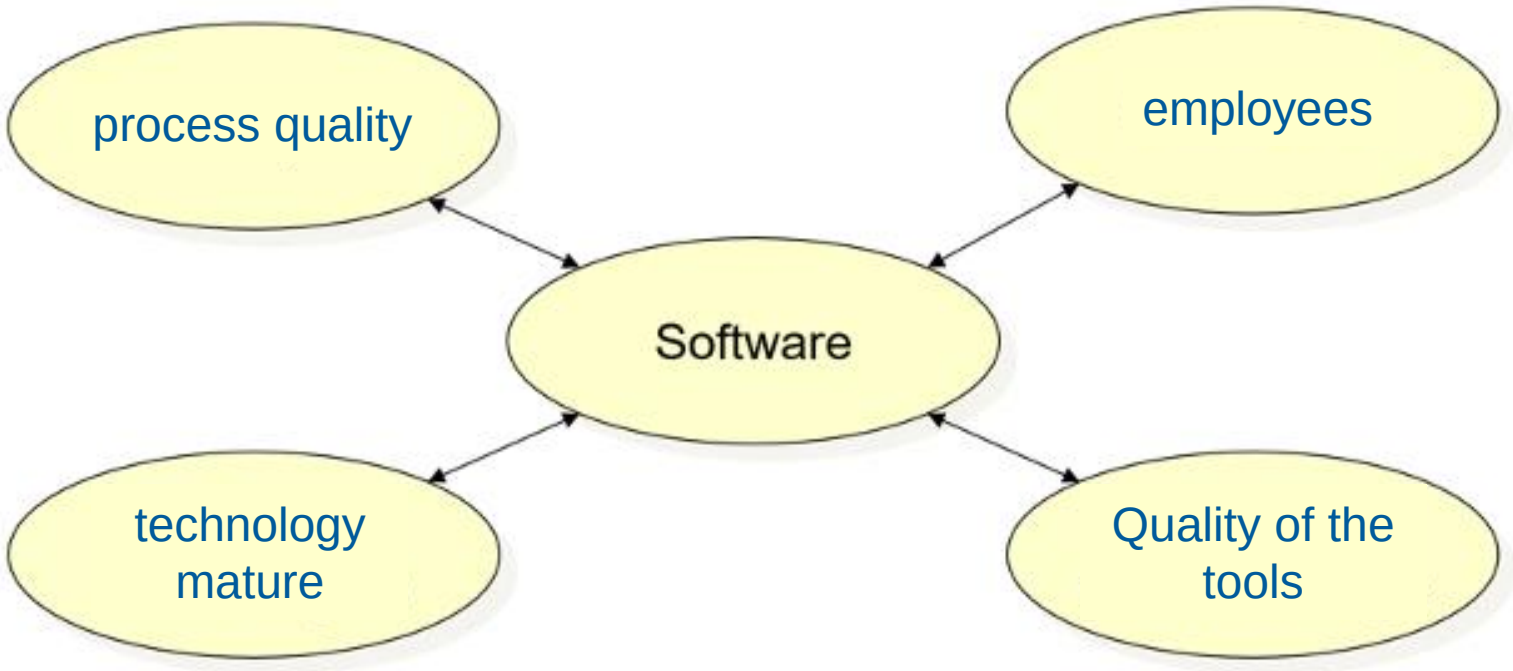
Main goal

- The main goal of software engineering is the development of high-quality software products.
- Errors prevent this goal from being achieved.
- Note: It is practically impossible to develop error-free software with a reasonable effort!

- **Definition of error**:
  - We understand an error to be the deviation of a determined, observed or measured value, state or behavior of a software product from the corresponding specified, theoretically correct or considered correct value, state or behavior.

How high-quality software can be developed?

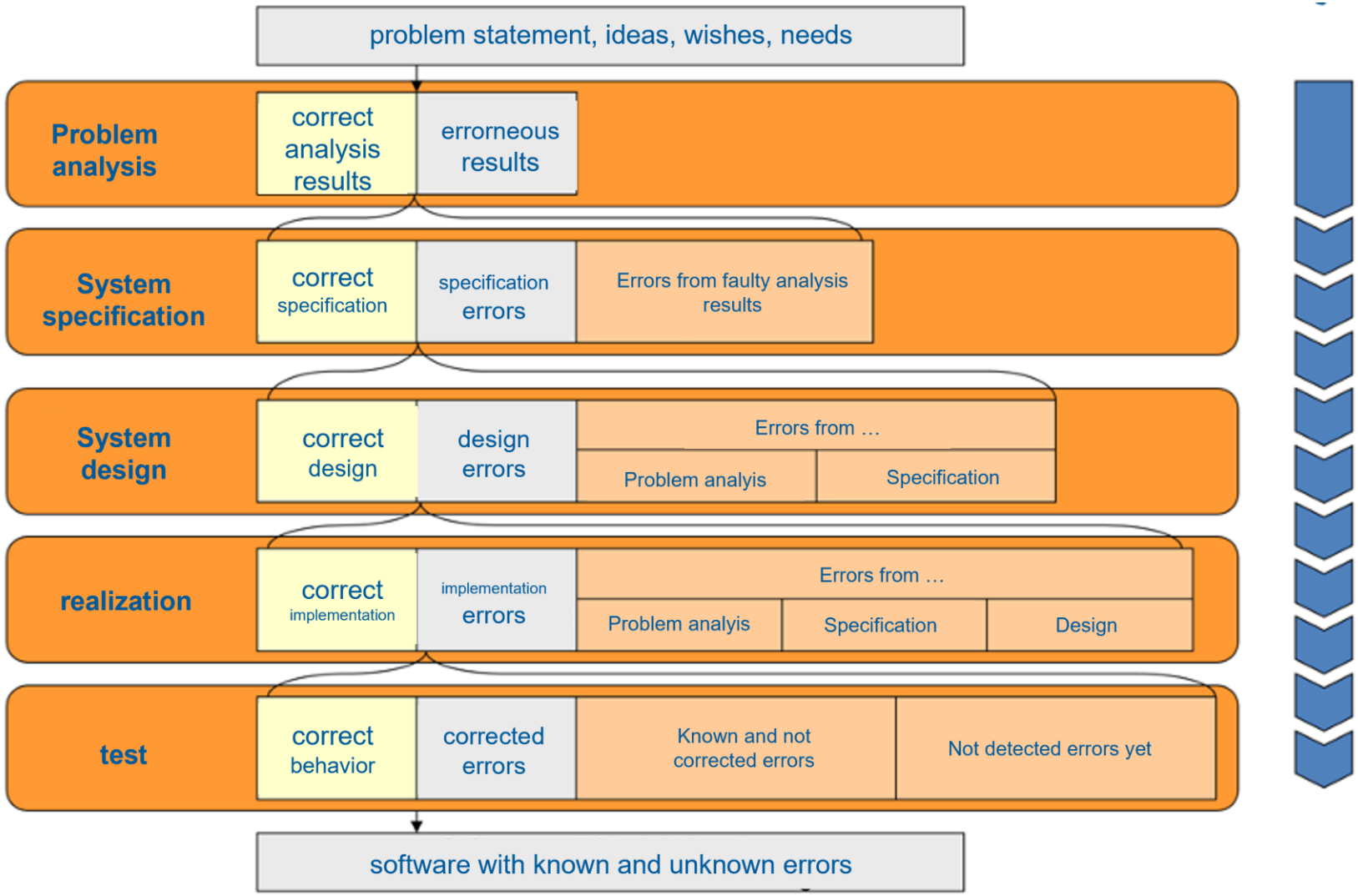
- Factors influencing the software quality



Quality Defects

- The causes of quality defects often lie in the early phases of a project.
- Early errors induce errors in subsequent project steps and the individual errors add up.
- In conclusion, it can be said that errors should be detected as early as possible.
- This keeps the costs for error diagnosis and correction within reasonable limits.

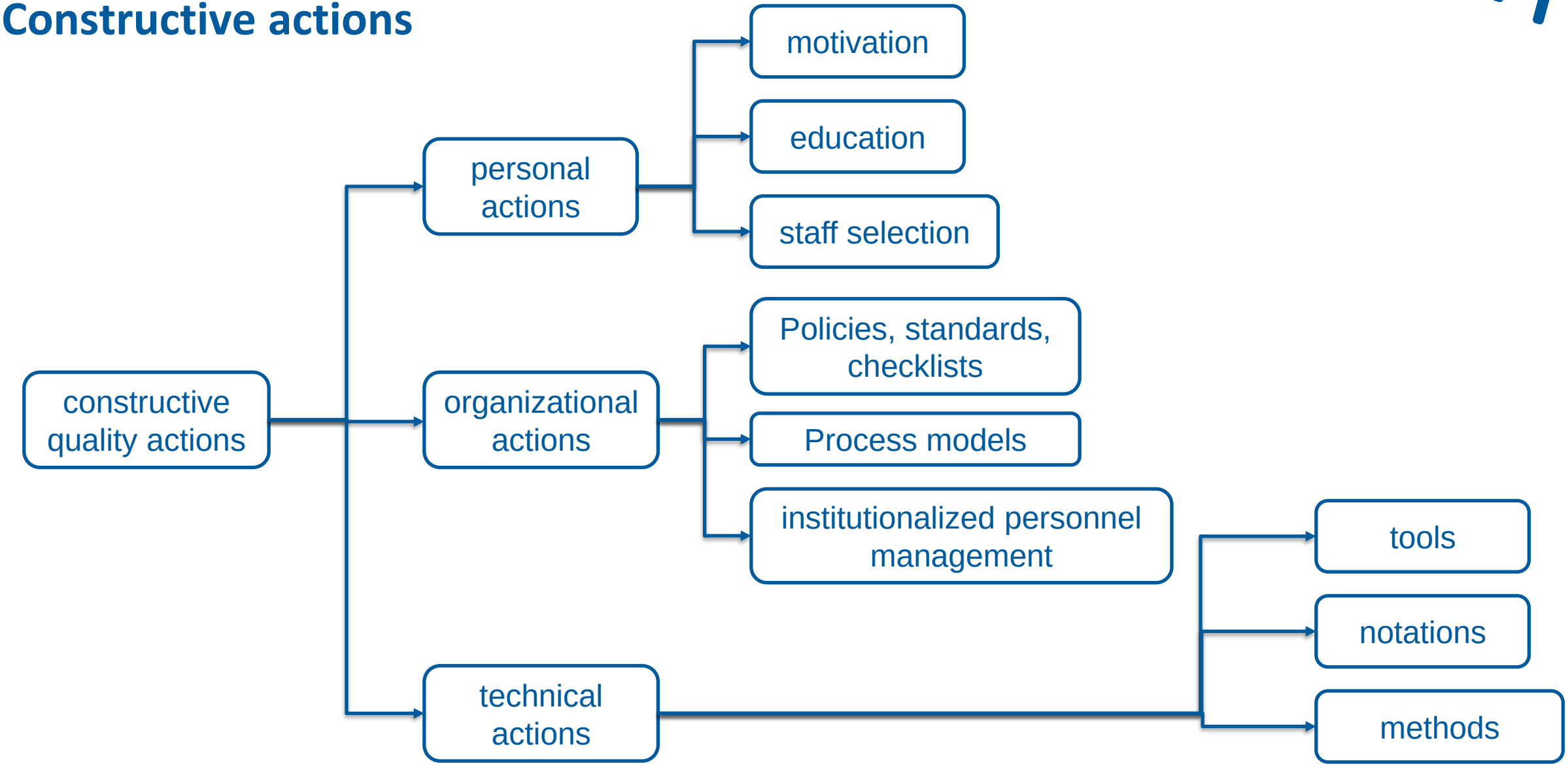
How high-quality software can be developed?



Constructive actions

- Ensure that the creation process and the created product have certain (quality) characteristics
- They include technical, organizational and personnel measures.

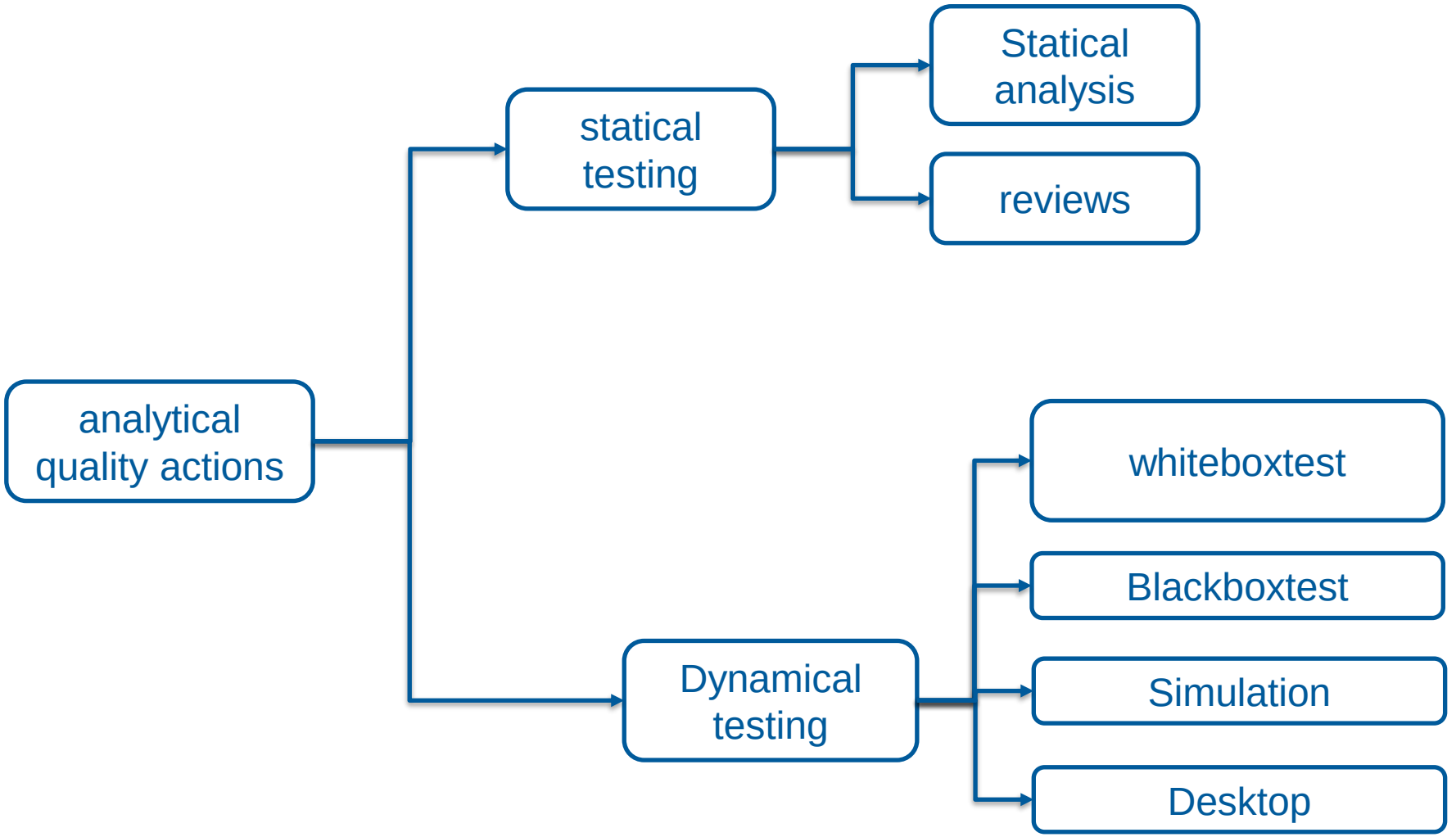
Constructive actions



Analytical actions

- These are measures for checking or assessing the actual quality of a product.
- They do not result in better quality generally, but the lead to further quality improvement measures.

Analytical actions



2.4 Exercises

Definition of quality

- Which statement fits best?

- The quality of a software is determined by ...
  1. The satisfaction of software users.
  2. All properties of the software in relation to the fulfillment of defined requirements.
  3. 100% error-free tests.

Software quality in practice

- Which statement fits best?

- In practice, the quality of software is determined by ...
  1. measurable quality indicators. Each indicator determines the fulfillment of a sub-characteristic.
  2. a large number of tests that have to be passed without errors.
  3. Test phases by future users as part of the acceptance process.

Quality features according to ISO 25010

- Which statement fits best?

- Select the 8 quality characteristic groups according to ISO 25010!
  1. Functionality, portability, maintainability, efficiency, compatibility, reliability, usability, security
  2. Fault tolerance, stability, time behavior, maturity, traceability, security, confidentiality, integrity
  3. Adaptability, testability, resource consumption, interoperability, traceability, security, maturity, fault tolerance

Quality characteristics can be in contrast to each other

- Why are resource consumption and time efficiency typically in opposition to each other?
  1. Time efficiency is often achieved by storing redundant information. Accordingly, more memory is required to increase performance.
  2. The software runs significantly faster due to lower resource consumption.
  3. The two quality features are not mutually exclusive!

Definition of error

- Which statement describes the term best?
  1. An error occurs when the software crashes.
  2. An error occurs when the software is in an infinite loop.
  3. An error occurs if the software behaves contrary to the specification.

Factors influencing software quality

- What influences software quality?
  1. Use of the currently most popular programming language
  2. Employees in the project team
  3. Tool quality
  4. Quality of the software development process
  5. Maturity of the technologies used
  6. Development with NoSQL databases

How do you avoid costs due to quality defects?

- Which statements are correct?
  1. Errors should be detected as early as possible in order to avoid subsequent errors.
  2. In any case, the software is delivered quickly (time to market!) and the errors are then rectified during operation.
  3. It is sufficient to test well before delivery and correct the errors.

Constructive actions...

- How does the sentence continue correctly?
- Constructive measures...
  1. ... provide assistance in coding the software.
  2. ... ensure that the development process and the software have a certain quality.
  3. ... are measures that relate to the programming (“construction”) of the software.

Analytical actions...

- How does the sentence continue correctly?
  1. ... are measures that lead to improved quality by examining the software.
  2. ... provide assistance in analyzing the software.
  3. ... are measures to determine the current software quality. They do not result in better quality but lead to further quality improvements.